Advanced Topics - Zero-Copy, Load Balancing, and Production Patterns

Theory, Edge Cases & Code Examples

- ✓ Overview
- ✓ Zero-Copy I/O: Eliminating Memory Copies
- ✓ Kernel-Level File Transfer
- ✓ Zero-Copy Between FDs
- ✓ Memory-Mapped I/O

→ Swipe to learn →

Advanced Topics - Zero-Copy, Load Balancing, and Production Patterns

Theory

📌 Overview

After mastering socket fundamentals and I/O multiplexing (select, poll, epoll), production network applications require advanced patterns for scalability, reliability, and performance. This topic covers real-world architectural patterns and optimizations used in high-performance servers.

What You'll Learn:

1. Zero-copy I/O techniques (sendfile, splice, mmap)
2. Reactor vs Proactor architectural patterns
3. Connection pooling and management
4. Load balancing strategies
5. Protocol design best practices
6. Flow control and backpressure
7. Graceful degradation under load
8. Production-grade error handling
9. Performance monitoring and metrics

Advanced Topics - Zero-Copy, Load Balancing, and Production Patterns

Theory

📌 1. Zero-Copy I/O: Eliminating Memory Copies

The Problem: Traditional I/O Copies Data Multiple Times

```
Sending a file to a socket (traditional approach):

Disk → Kernel buffer → User buffer → Kernel socket buffer → Network
       [DMA copy]     [CPU copy]    [CPU copy]

3 copies total: 1 DMA + 2 CPU
Context switches: 4 (read syscall enter/exit, write syscall enter/exit)
```

Traditional Code (Inefficient):

```
// ❌ SLOW: Multiple copies
char buf[4096];
int file_fd = open("file.txt", O_RDONLY);
int socket_fd = accept(...);

while (true) {
    int n = read(file_fd, buf, sizeof(buf));  // Copy 1: Kernel → User
    if (n <= 0) break;

    send(socket_fd, buf, n, 0);  // Copy 2: User → Kernel
}

// Total: 2 CPU copies through userspace
```

Zero-Copy Solutions:

Advanced Topics - Zero-Copy, Load Balancing, and Production Patterns

Theory

📌 1.1 sendfile() - Kernel-Level File Transfer (1/3)

Concept: Transfer data directly from file to socket without passing through userspace.

```
Disk → Kernel buffer → Kernel socket buffer → Network
       [DMA copy]       [DMA copy or CPU copy]

2 copies total (or 0 with DMA gather), no userspace copies
Context switches: 1 (sendfile syscall enter/exit)
```

API:

```
#include <sys/sendfile.h>

ssize_t sendfile(int out_fd, int in_fd, off_t *offset, size_t count);
```

Parameters:

- `out_fd` : Destination socket FD
- `in_fd` : Source file FD
- `offset` : Start position in file (NULL = from current position)
- `count` : Number of bytes to transfer

Advanced Topics - Zero-Copy, Load Balancing, and Production Patterns

Theory

📌 1.1 sendfile() - Kernel-Level File Transfer (2/3)

Example:

```
#include <sys/sendfile.h>
#include <sys/stat.h>
#include <fcntl.h>

void send_file_zero_copy(int socket_fd, const char *filepath) {
    int file_fd = open(filepath, O_RDONLY);
    if (file_fd < 0) {
        perror("open");
        return;
    }

    // Get file size
    struct stat st;
    fstat(file_fd, &st);
    off_t file_size = st.st_size;

    // Send entire file with zero-copy
    off_t offset = 0;
    ssize_t sent = sendfile(socket_fd, file_fd, &offset, file_size);

    if (sent < 0) {
        perror("sendfile");
    } else {
        std::cout << "Sent " << sent << " bytes using zero-copy\n";
    }

    close(file_fd);
}
```

Advanced Topics - Zero-Copy, Load Balancing, and Production Patterns

Theory

📌 1.1 sendfile() - Kernel-Level File Transfer (3/3)

Performance Comparison:

```
File size: 100 MB

Traditional read()/send():
- CPU copies: 200 MB (100 MB × 2)
- CPU usage: ~60%
- Throughput: ~400 MB/s
- Time: 250 ms

sendfile():
- CPU copies: 0 MB (kernel handles it)
- CPU usage: ~5%
- Throughput: ~1200 MB/s
- Time: 83 ms

Speedup: 3x faster
```

Limitations:

1. **Output must be socket:** Can't use sendfile() to copy file→file
2. **Input must be file:** Can't use with pipes or sockets
3. **No modification:** Can't modify data in transit
4. **Linux-specific behavior:** Other OSes have different semantics

---

Advanced Topics - Zero-Copy, Load Balancing, and Production Patterns

Theory

📌 1.2 splice() - Zero-Copy Between FDs (1/3)

Concept: Move data between two file descriptors through a kernel pipe.

```
#include <fcntl.h>

ssize_t splice(int fd_in, loff_t *off_in,
               int fd_out, loff_t *off_out,
               size_t len, unsigned int flags);
```

Flags:

- `SPLICE_F_MOVE` : Move pages instead of copying (if possible)
- `SPLICE_F_NONBLOCK` : Non-blocking operation
- `SPLICE_F_MORE` : More data coming (hint for TCP_CORK)

Use Cases:

1. **File → Socket** (like sendfile, but more flexible)
2. **Socket → File** (receive data to file)
3. **Socket → Socket** (proxy data)
4. **Pipe → Anything** (intermediate buffering)

Advanced Topics - Zero-Copy, Load Balancing, and Production Patterns

Theory

📌 1.2 splice() - Zero-Copy Between FDs (2/3)

Example: HTTP File Server with splice()

```
#include <fcntl.h>

void serve_file_with_splice(int client_fd, const char *filepath) {
    int file_fd = open(filepath, O_RDONLY);
    if (file_fd < 0) {
        perror("open");
        return;
    }

    // Get file size
    struct stat st;
    fstat(file_fd, &st);
    off_t remaining = st.st_size;

    // Create pipe for splice
    int pipe_fds[2];
    if (pipe(pipe_fds) < 0) {
        perror("pipe");
        close(file_fd);
        return;
    }

    const size_t CHUNK_SIZE = 65536;  // 64 KB

    while (remaining > 0) {
        size_t to_splice = std::min((size_t)remaining, CHUNK_SIZE);

        // splice: file → pipe
        ssize_t n1 = splice(file_fd, nullptr, pipe_fds[1], nullptr,
                            to_splice, SPLICE_F_MOVE | SPLICE_F_MORE);
        if (n1 <= 0) break;

        // splice: pipe → socket
        ssize_t n2 = splice(pipe_fds[0], nullptr, client_fd, nullptr,
                            n1, SPLICE_F_MOVE | SPLICE_F_MORE);
        if (n2 <= 0) break;

        remaining -= n2;
    }
// ...continues on next slide
```

📌 1.2 splice() - Zero-Copy Between FDs (3/3)

[Code continues across slides]

```
// ...continued from previous slide

    close(pipe_fds[0]);
    close(pipe_fds[1]);
    close(file_fd);

    std::cout << "Transferred " << st.st_size << " bytes with splice\n";
}
```

Advantages over sendfile():

1. **More flexible:** Works with pipes, sockets, files
2. **Bidirectional:** Can receive data too (socket → file)
3. **Composable:** Can chain multiple splice() calls
4. **Non-blocking:** Supports SPLICE_F_NONBLOCK

Advanced Topics - Zero-Copy, Load Balancing, and Production Patterns

Theory

📌 1.3 mmap() - Memory-Mapped I/O (1/2)

Concept: Map file contents directly into process address space. No read()/write() needed.

```
#include <sys/mman.h>

void *mmap(void *addr, size_t length, int prot, int flags,
           int fd, off_t offset);
```

Example: Serve File with mmap() + send()

```
#include <sys/mman.h>

void serve_file_with_mmap(int socket_fd, const char *filepath) {
    int file_fd = open(filepath, O_RDONLY);
    if (file_fd < 0) {
        perror("open");
        return;
    }

    // Get file size
    struct stat st;
    fstat(file_fd, &st);
    size_t file_size = st.st_size;

    // Map file into memory
    void *file_data = mmap(NULL, file_size, PROT_READ, MAP_PRIVATE, file_fd, 0);
    if (file_data == MAP_FAILED) {
        perror("mmap");
        close(file_fd);
        return;
    }

    // Send mapped memory to socket
    ssize_t sent = send(socket_fd, file_data, file_size, 0);

    std::cout << "Sent " << sent << " bytes from mmap'd file\n";

    // Cleanup
    munmap(file_data, file_size);
    close(file_fd);
}
```

📌 1.3 mmap() - Memory-Mapped I/O (2/2)

When to Use mmap():

Scenario	Best Choice
Send entire file to socket	`sendfile()` (fastest)
Send file with epoll	`splice()` (non-blocking)
Random access to file	`mmap()` (direct access)
Multiple reads from same file	`mmap()` (cached in memory)
Modify file in place	`mmap()` with `PROT_WRITE`
Share memory between processes	`mmap()` with `MAP_SHARED`

Gotchas:

```
// ❌ WRONG: File can be truncated while mapped
void *data = mmap(..., file_size, ...);
// Another process: truncate(file, 0);
char c = ((char*)data)[100];  // ❌ SEGFAULT! File is now empty

// ✅ CORRECT: Lock file before mmap
flock(file_fd, LOCK_SH);  // Shared lock
void *data = mmap(...);
// Use data
munmap(data, size);
flock(file_fd, LOCK_UN);  // Unlock
```

Advanced Topics - Zero-Copy, Load Balancing, and Production Patterns

Theory

📌 2. Architectural Patterns: Reactor vs Proactor

Two fundamental patterns for asynchronous I/O:

Advanced Topics - Zero-Copy, Load Balancing, and Production Patterns

Theory

📌 2.1 Reactor Pattern (Event-Driven) (1/3)

Concept: "Tell me when FD is ready, I'll do the I/O."

```
Application registers interest in events
      ↓
Event loop (epoll_wait) blocks
      ↓
Event occurs (data available)
      ↓
Event loop wakes up
      ↓
Application calls recv()/send() ← Application does I/O
```

Characteristics:

- **Reactive:** Application reacts to "ready" events
- **Synchronous I/O:** Application calls recv()/send() itself
- **Examples:** epoll, kqueue, poll, select
- **Blocking point:** recv()/send() (even if non-blocking, CPU waits)

Advanced Topics - Zero-Copy, Load Balancing, and Production Patterns

Theory

📌 2.1 Reactor Pattern (Event-Driven) (2/3)

Reactor Implementation:

```
class Reactor {
private:
    int epfd;
    std::unordered_map<int, std::function<void(uint32_t)>> handlers;

public:
    Reactor() {
        epfd = epoll_create1(EPOLL_CLOEXEC);
    }

    // Register FD with callback
    void register_handler(int fd, uint32_t events,
                          std::function<void(uint32_t)> handler) {
        handlers[fd] = handler;

        struct epoll_event ev;
        ev.events = events;
        ev.data.fd = fd;
        epoll_ctl(epfd, EPOLL_CTL_ADD, fd, &ev);
    }

    // Event loop
    void run() {
        struct epoll_event events[128];

        while (true) {
            int n = epoll_wait(epfd, events, 128, -1);

            for (int i = 0; i < n; i++) {
                int fd = events[i].data.fd;
                uint32_t ev = events[i].events;

                // Call registered handler
                if (handlers.count(fd)) {
                    handlers[fd](ev);
                }
            }
        }
    }
// ...continues on next slide
```

Advanced Topics - Zero-Copy, Load Balancing, and Production Patterns

Theory

📌 2.1 Reactor Pattern (Event-Driven) (3/3)

[Code continues across slides]

```
// ...continued from previous slide
};

// Usage
Reactor reactor;

// Accept handler
reactor.register_handler(listen_fd, EPOLLIN, [&](uint32_t events) {
    int client = accept(listen_fd, nullptr, nullptr);

    // Register client handler
    reactor.register_handler(client, EPOLLIN, [client](uint32_t events) {
        char buf[1024];
        int n = recv(client, buf, sizeof(buf), 0);  // ← Reactor: App does I/O

        if (n <= 0) {
            close(client);
        } else {
            send(client, buf, n, 0);
        }
    });
});

reactor.run();
```

Pros:
- ✅ Simple to understand
- ✅ Direct control over I/O operations
- ✅ Widely supported (epoll, kqueue)
- ✅ Easy to debug

Cons:
- ❌ Application still blocks on recv()/send()
- ❌ CPU cycles spent waiting for I/O completion
- ❌ Doesn't scale to hundreds of thousands of connections

Advanced Topics - Zero-Copy, Load Balancing, and Production Patterns

Theory

📌 2.2 Proactor Pattern (Asynchronous I/O) (1/4)

Concept: "Start the I/O for me, tell me when it's done."

```
Application initiates async I/O (io_uring, IOCP)
      ↓
Kernel performs I/O in background ← Kernel does I/O
      ↓
I/O completes
      ↓
Kernel notifies application
      ↓
Application processes completed I/O (data already in buffer)
```

Characteristics:

- **Proactive:** Application initiates I/O operations
- **Asynchronous I/O:** Kernel handles I/O completion
- **Examples:** io_uring (Linux), IOCP (Windows), AIO (POSIX)
- **Blocking point:** None - CPU can do other work while I/O happens

Advanced Topics - Zero-Copy, Load Balancing, and Production Patterns

Theory

📌 2.2 Proactor Pattern (Asynchronous I/O) (2/4)

Proactor Implementation (io_uring):

```
#include <liburing.h>

class Proactor {
private:
    io_uring ring;

public:
    Proactor(int queue_depth = 256) {
        io_uring_queue_init(queue_depth, &ring, 0);
    }

    // Submit async read
    void async_read(int fd, char *buf, size_t len,
                    std::function<void(int)> callback) {
        // Get submission queue entry
        io_uring_sqe *sqe = io_uring_get_sqe(&ring);

        // Prepare read operation
        io_uring_prep_read(sqe, fd, buf, len, 0);

        // Store callback in user_data
        io_uring_sqe_set_data(sqe, new auto(callback));

        // Submit to kernel (non-blocking)
        io_uring_submit(&ring);  // ← Kernel will do I/O asynchronously
    }

    // Process completed I/O
    void run() {
        io_uring_cqe *cqe;

        while (true) {
            // Wait for completion
            io_uring_wait_cqe(&ring, &cqe);

            // Extract callback
            auto callback = (std::function<void(int)>*)io_uring_cqe_get_data(cqe);

            // Call callback with result
// ...continues on next slide
```

📌 2.2 Proactor Pattern (Asynchronous I/O) (3/4)

[Code continues across slides]

```
// ...continued from previous slide
            (*callback)(cqe->res);  // ← Data already read by kernel!

            delete callback;
            io_uring_cqe_seen(&ring, cqe);
        }
    }
};

// Usage
Proactor proactor;

// Accept and start async read
int client = accept(listen_fd, nullptr, nullptr);
char *buf = new char[1024];

proactor.async_read(client, buf, 1024, [client, buf](int bytes_read) {
    // ← This callback runs AFTER kernel completes read
    // Data is already in buf!

    if (bytes_read > 0) {
        send(client, buf, bytes_read, 0);
    }

    delete[] buf;
});

proactor.run();
```

📌 2.2 Proactor Pattern (Asynchronous I/O) (4/4)

Pros:

- ✅ True asynchronous I/O (kernel does the work)
- ✅ CPU can process other tasks while I/O happens
- ✅ Scales to millions of connections
- ✅ Works with files (unlike epoll)

Cons:

- ❌ More complex API
- ❌ Linux 5.1+ required (io_uring)
- ❌ Harder to debug
- ❌ Not portable (Windows uses IOCP, different API)

Advanced Topics - Zero-Copy, Load Balancing, and Production Patterns

Theory

📌 2.3 Reactor vs Proactor Comparison

Aspect	Reactor (epoll)	Proactor (io_uring)
I/O Model	Synchronous (app calls recv)	Asynchronous (kernel calls recv)
CPU Usage	Blocks during recv/send	Free during I/O
Scalability	Good (100K connections)	Excellent (1M+ connections)
File I/O	❌ Doesn't work	✅ Works
Complexity	Medium	High
Portability	Linux, BSD, macOS	Linux 5.1+ only
Learning Curve	Easier	Harder
Best For	Most network servers	Ultra-high-performance servers

When to Use:

- **Reactor (epoll):**
- Most web servers, APIs, chat servers
- < 100,000 connections
- Linux 2.6+ (20 years old)
- **Proactor (io_uring):**
- CDN edge servers, database servers
- 100,000 connections
- Mixed file + socket I/O
- Linux 5.1+ (2019+)

Advanced Topics - Zero-Copy, Load Balancing, and Production Patterns

Theory

📌 3. Connection Pooling (1/4)

Problem: Creating TCP connections is expensive (3-way handshake, slow start).

```
Without pooling:
Request 1: connect() [100ms] → send() → recv() → close()
Request 2: connect() [100ms] → send() → recv() → close()
Request 3: connect() [100ms] → send() → recv() → close()

Total: 300ms overhead just for connections
```

Solution: Reuse connections across multiple requests.

```
With pooling:
Startup: connect() [100ms] ← One-time cost
Request 1: send() → recv() [reuse connection]
Request 2: send() → recv() [reuse connection]
Request 3: send() → recv() [reuse connection]

Total: 100ms overhead (3x improvement)
```

Advanced Topics - Zero-Copy, Load Balancing, and Production Patterns

Theory

📌 3. Connection Pooling (2/4)

Connection Pool Implementation:

```
class ConnectionPool {
private:
    std::string host;
    int port;

    std::queue<int> available;  // Available connections
    std::set<int> in_use;       // Connections being used
    std::mutex pool_mutex;

    int max_connections = 10;
    int idle_timeout_sec = 60;

    std::unordered_map<int, std::chrono::steady_clock::time_point> last_used;

public:
    ConnectionPool(const std::string& h, int p, int max_conns = 10)
        : host(h), port(p), max_connections(max_conns) {}

    // Get connection from pool
    int acquire() {
        std::lock_guard<std::mutex> lock(pool_mutex);

        // Check if idle connection available
        if (!available.empty()) {
            int fd = available.front();
            available.pop();
            in_use.insert(fd);
            return fd;
        }

        // Create new connection if under limit
        if (in_use.size() < max_connections) {
            int fd = create_connection();
            in_use.insert(fd);
            return fd;
        }

        // Pool exhausted
        return -1;  // Caller should retry or wait
// ...continues on next slide
```

📌 3. Connection Pooling (3/4)

```
// ...continued from previous slide
    }

    // Return connection to pool
    void release(int fd) {
        std::lock_guard<std::mutex> lock(pool_mutex);

        in_use.erase(fd);
        available.push(fd);
        last_used[fd] = std::chrono::steady_clock::now();
    }

    // Remove stale connections
    void cleanup_idle() {
        std::lock_guard<std::mutex> lock(pool_mutex);

        auto now = std::chrono::steady_clock::now();

        while (!available.empty()) {
            int fd = available.front();

            if (now - last_used[fd] > std::chrono::seconds(idle_timeout_sec)) {
                available.pop();
                close(fd);
                last_used.erase(fd);
            } else {
                break;  // Queue is time-ordered
            }
        }
    }

private:
    int create_connection() {
        int fd = socket(AF_INET, SOCK_STREAM, 0);

        sockaddr_in addr{};
        addr.sin_family = AF_INET;
        addr.sin_port = htons(port);
        inet_pton(AF_INET, host.c_str(), &addr.sin_addr);

        if (connect(fd, (sockaddr*)&addr, sizeof(addr)) < 0) {
// ...continues on next slide
```

📌 3. Connection Pooling (4/4)

[Code continues across slides]

```
// ...continued from previous slide
            close(fd);
            return -1;
        }

        return fd;
    }
};

// Usage
ConnectionPool pool("api.example.com", 443, 20);

// Use connection
int fd = pool.acquire();
if (fd >= 0) {
    send(fd, "GET /api HTTP/1.1\r\n\r\n", 20, 0);

    char buf[4096];
    recv(fd, buf, sizeof(buf), 0);

    pool.release(fd);  // Return to pool (doesn't close)
}

// Periodic cleanup
pool.cleanup_idle();
```

Advanced Features:

1. **Health Checks:** Ping connections before returning them
2. **Automatic Retry:** Reconnect if connection is stale
3. **Load Balancing:** Pool across multiple backend servers
4. **Metrics:** Track pool utilization, wait times
5. **Circuit Breaker:** Disable pool if backend is down

Advanced Topics - Zero-Copy, Load Balancing, and Production Patterns

Theory

📌 4. Load Balancing Strategies

Problem: Single backend server can't handle all traffic. Need to distribute load across multiple servers.

Advanced Topics - Zero-Copy, Load Balancing, and Production Patterns

Theory

📌 4.1 Round-Robin Load Balancing (1/2)

Concept: Distribute requests evenly across servers in sequential order.

```
Request 1 → Server A
Request 2 → Server B
Request 3 → Server C
Request 4 → Server A (wrap around)
```

Implementation:

```
class RoundRobinLoadBalancer {
private:
    std::vector<std::string> backends;
    std::atomic<size_t> current{0};

public:
    RoundRobinLoadBalancer(std::vector<std::string> servers)
        : backends(std::move(servers)) {}

    std::string next_server() {
        size_t index = current.fetch_add(1, std::memory_order_relaxed) % backends.size();
        return backends[index];
    }
};

// Usage
RoundRobinLoadBalancer lb({"server1.com", "server2.com", "server3.com"});

for (int i = 0; i < 10; i++) {
    std::string server = lb.next_server();
    std::cout << "Request " << i << " → " << server << "\n";
}

// Output:
// Request 0 → server1.com
// Request 1 → server2.com
// Request 2 → server3.com
// Request 3 → server1.com
// ...
```

Advanced Topics - Zero-Copy, Load Balancing, and Production Patterns

Theory

📌 4.1 Round-Robin Load Balancing (2/2)

Pros:

- ✅ Simple to implement
- ✅ Fair distribution
- ✅ No state needed

Cons:

- ❌ Doesn't account for server load
- ❌ Doesn't account for request complexity
- ❌ Dead server still gets traffic

Advanced Topics - Zero-Copy, Load Balancing, and Production Patterns

Theory

📌 4.2 Weighted Round-Robin (1/3)

Concept: Give more traffic to more powerful servers.

```
Server A: weight=3 (3× capacity)
Server B: weight=2 (2× capacity)
Server C: weight=1 (1× capacity)

Sequence: A, A, A, B, B, C (repeat)
```

Advanced Topics - Zero-Copy, Load Balancing, and Production Patterns

Theory

📌 4.2 Weighted Round-Robin (2/3)

Implementation:

```
class WeightedRoundRobinLoadBalancer {
private:
    struct Backend {
        std::string host;
        int weight;
        int current_weight;
    };

    std::vector<Backend> backends;
    std::mutex lb_mutex;

public:
    WeightedRoundRobinLoadBalancer(std::vector<std::pair<std::string, int>> servers) {
        for (const auto& [host, weight] : servers) {
            backends.push_back({host, weight, 0});
        }
    }

    std::string next_server() {
        std::lock_guard<std::mutex> lock(lb_mutex);

        int total_weight = 0;
        Backend *selected = nullptr;

        for (auto& backend : backends) {
            backend.current_weight += backend.weight;
            total_weight += backend.weight;

            if (!selected || backend.current_weight > selected->current_weight) {
                selected = &backend;
            }
        }

        if (selected) {
            selected->current_weight -= total_weight;
            return selected->host;
        }

        return backends[0].host;  // Fallback
// ...continues on next slide
```

📌 4.2 Weighted Round-Robin (3/3)

[Code continues across slides]

```
// ...continued from previous slide
    }
};

// Usage
WeightedRoundRobinLoadBalancer lb({
    {"fast-server.com", 5},    // 5× capacity
    {"medium-server.com", 3},  // 3× capacity
    {"slow-server.com", 1}     // 1× capacity
});

// Outputs: fast, fast, fast, fast, fast, medium, medium, medium, slow
```

Advanced Topics - Zero-Copy, Load Balancing, and Production Patterns

Theory

📌 4.3 Least Connections (1/2)

Concept: Send traffic to server with fewest active connections.

```
class LeastConnectionsLoadBalancer {
private:
    struct Backend {
        std::string host;
        std::atomic<int> active_connections{0};
    };

    std::vector<Backend> backends;

public:
    LeastConnectionsLoadBalancer(std::vector<std::string> servers) {
        for (const auto& host : servers) {
            backends.push_back({host, 0});
        }
    }

    std::pair<std::string, int> acquire() {
        // Find backend with fewest connections
        Backend *selected = &backends[0];
        int min_conns = selected->active_connections;

        for (auto& backend : backends) {
            int conns = backend.active_connections;
            if (conns < min_conns) {
                selected = &backend;
                min_conns = conns;
            }
        }

        // Increment connection count
        int conn_id = selected->active_connections.fetch_add(1);

        return {selected->host, conn_id};
    }

    void release(const std::string& host) {
        for (auto& backend : backends) {
            if (backend.host == host) {
// ...continues on next slide
```

📌 4.3 Least Connections (2/2)

[Code continues across slides]

```
// ...continued from previous slide
                backend.active_connections.fetch_sub(1);
                break;
            }
        }
    }
};
```

Pros:

- ✅ Accounts for actual server load
- ✅ Better than round-robin for long-lived connections

Cons:

- ❌ Doesn't account for request complexity
- ❌ Requires state tracking

Advanced Topics - Zero-Copy, Load Balancing, and Production Patterns

Theory

📌 4.4 Consistent Hashing

Concept: Hash client ID to deterministically route to same server (session affinity).

```
Client A → hash("A") % 3 = 1 → Server B (always)
Client B → hash("B") % 3 = 0 → Server A (always)
Client C → hash("C") % 3 = 2 → Server C (always)
```

Use Case: Sessions, caching (user always hits same cache server).

```
class ConsistentHashLoadBalancer {
private:
    std::vector<std::string> backends;

public:
    ConsistentHashLoadBalancer(std::vector<std::string> servers)
        : backends(std::move(servers)) {}

    std::string get_server(const std::string& client_id) {
        size_t hash = std::hash<std::string>{}(client_id);
        size_t index = hash % backends.size();
        return backends[index];
    }
};

// Usage
ConsistentHashLoadBalancer lb({"cache1.com", "cache2.com", "cache3.com"});

// User "alice" always routes to same cache server
std::cout << lb.get_server("alice") << "\n";  // cache2.com
std::cout << lb.get_server("alice") << "\n";  // cache2.com (same)
std::cout << lb.get_server("bob") << "\n";    // cache1.com
```

Pros:
- ✅ Session affinity (user always hits same server)
- ✅ Cache efficiency (data stays on same server)

Cons:
- ❌ Uneven distribution if hash is poor
- ❌ Adding/removing servers changes all mappings

📌 5. Protocol Design Best Practices

Key Principles for Designing Network Protocols:

Advanced Topics - Zero-Copy, Load Balancing, and Production Patterns

Theory

📌 5.1 Fixed vs Variable-Length Messages (1/2)

Fixed-Length:

```
[Command: 1 byte][Length: 4 bytes][Payload: N bytes]
```

Pros:
- ✅ Easy to parse (no delimiter searching)
- ✅ Predictable buffer sizes

Cons:
- ❌ Wastes space for small messages
- ❌ Limits maximum message size

Variable-Length:

```
[Length: 4 bytes][Payload: variable]
```

Pros:
- ✅ Efficient for varying message sizes
- ✅ No maximum limit

Cons:
- ❌ Must read length first, then payload (2 recv calls)
- ❌ More complex parsing

Advanced Topics - Zero-Copy, Load Balancing, and Production Patterns

Theory

📌 5.1 Fixed vs Variable-Length Messages (2/2)

Best Practice: Length-Prefixed Messages

```
struct MessageHeader {
    uint32_t length;  // Big-endian (network byte order)
    uint16_t type;
    uint16_t flags;
};

// Sender
void send_message(int fd, uint16_t type, const std::string& payload) {
    MessageHeader header;
    header.length = htonl(sizeof(header) + payload.size());
    header.type = htons(type);
    header.flags = 0;

    // Send header
    send(fd, &header, sizeof(header), 0);

    // Send payload
    send(fd, payload.data(), payload.size(), 0);
}

// Receiver
std::string recv_message(int fd, uint16_t& type) {
    // Read header
    MessageHeader header;
    recv(fd, &header, sizeof(header), MSG_WAITALL);

    uint32_t length = ntohl(header.length) - sizeof(header);
    type = ntohs(header.type);

    // Read payload
    std::string payload(length, '\0');
    recv(fd, payload.data(), length, MSG_WAITALL);

    return payload;
}
```

Advanced Topics - Zero-Copy, Load Balancing, and Production Patterns

Theory

📌 5.2 Byte Order (Endianness)

Problem: Different CPUs store multi-byte integers differently.

- **Little-Endian** (x86): 0x12345678 stored as [78 56 34 12]
- **Big-Endian** (Network): 0x12345678 stored as [12 34 56 78]

Solution: Always use network byte order (big-endian)

```
// Send
uint32_t value = 12345;
uint32_t net_value = htonl(value);  // Host to network long
send(fd, &net_value, sizeof(net_value), 0);

// Receive
uint32_t net_value;
recv(fd, &net_value, sizeof(net_value), 0);
uint32_t value = ntohl(net_value);  // Network to host long
```

Functions:

- `htons()`: Host to network short (16-bit)
- `htonl()`: Host to network long (32-bit)
- `ntohs()`: Network to host short
- `ntohl()`: Network to host long

Advanced Topics - Zero-Copy, Load Balancing, and Production Patterns

Theory

📌 5.3 Version Negotiation

Problem: Protocol evolves over time. Need to support multiple versions.

Pattern: Version in Handshake

```
struct Handshake {
    uint8_t magic[4] = {'M', 'Y', 'P', 'X'};  // Protocol identifier
    uint16_t version_major = 1;
    uint16_t version_minor = 5;
};

// Server
void handle_handshake(int client_fd) {
    Handshake hs;
    recv(client_fd, &hs, sizeof(hs), MSG_WAITALL);

    if (memcmp(hs.magic, "MYPX", 4) != 0) {
        std::cerr << "Invalid protocol\n";
        close(client_fd);
        return;
    }

    uint16_t client_ver = ntohs(hs.version_major);

    if (client_ver > SERVER_VERSION_MAJOR) {
        std::cerr << "Client version too new\n";
        close(client_fd);
        return;
    }

    // Respond with server version
    Handshake response;
    response.version_major = htons(SERVER_VERSION_MAJOR);
    response.version_minor = htons(SERVER_VERSION_MINOR);
    send(client_fd, &response, sizeof(response), 0);

    std::cout << "Negotiated version: " << client_ver << "\n";
}
```

Advanced Topics - Zero-Copy, Load Balancing, and Production Patterns

Theory

📌 5.4 Error Handling in Protocols

Include Error Codes in Protocol

```
enum class ErrorCode : uint16_t {
    OK = 0,
    INVALID_REQUEST = 400,
    UNAUTHORIZED = 401,
    NOT_FOUND = 404,
    INTERNAL_ERROR = 500,
    SERVICE_UNAVAILABLE = 503
};

struct Response {
    uint32_t request_id;
    uint16_t error_code;
    uint16_t payload_length;
    // Payload follows
};

void send_error(int fd, uint32_t request_id, ErrorCode code, const std::string& message) {
    Response resp;
    resp.request_id = htonl(request_id);
    resp.error_code = htons(static_cast<uint16_t>(code));
    resp.payload_length = htons(message.size());

    send(fd, &resp, sizeof(resp), 0);
    send(fd, message.data(), message.size(), 0);
}
```

Advanced Topics - Zero-Copy, Load Balancing, and Production Patterns

Theory

📌 6. Flow Control and Backpressure

Problem: Fast producer overwhelms slow consumer.

```
Producer (fast) → [Buffer] → Consumer (slow)
                      ↑
                  Overflow! ❌
```

Advanced Topics - Zero-Copy, Load Balancing, and Production Patterns

Theory

📌 6.1 TCP Built-in Flow Control

TCP has automatic flow control via **receive window**:

```
Sender: "I want to send 10 KB"
Receiver: "My buffer only has 2 KB free, send 2 KB only"
Sender: Blocks until receiver consumes data
```

Problem: This blocks the sender, wasting resources.

📌 6.2 Application-Level Backpressure

Strategy 1: Stop Reading from Source

```
// epoll-based backpressure
if (write_queue[client_fd].size() > MAX_QUEUE_SIZE) {
    // Stop monitoring EPOLLIN on upstream source
    struct epoll_event ev;
    ev.events = EPOLLOUT;  // Only monitor writes
    ev.data.fd = upstream_fd;
    epoll_ctl(epfd, EPOLL_CTL_MOD, upstream_fd, &ev);

    std::cout << "Backpressure: paused reading from upstream\n";
}

// When write queue drains
if (write_queue[client_fd].empty()) {
    // Resume reading
    ev.events = EPOLLIN | EPOLLOUT;
    epoll_ctl(epfd, EPOLL_CTL_MOD, upstream_fd, &ev);

    std::cout << "Backpressure: resumed reading\n";
}
```

Strategy 2: Explicit Pause/Resume Messages

```
Consumer → Producer: PAUSE message (buffer full)
Producer: Stops sending data
Consumer: Processes data
Consumer → Producer: RESUME message (buffer has space)
Producer: Resumes sending
```

Strategy 3: Credit-Based Flow Control

```
Consumer → Producer: "You have 10 MB of credit"
Producer: Sends up to 10 MB
Producer → Consumer: 5 MB of data
Consumer: "You have 5 MB credit remaining"
```

Advanced Topics - Zero-Copy, Load Balancing, and Production Patterns

Theory

📌 7. Graceful Degradation Under Load

Problem: Server becomes overloaded during traffic spikes.

Strategies:

Advanced Topics - Zero-Copy, Load Balancing, and Production Patterns

Theory

📌 7.1 Connection Limits

```
const int MAX_CONNECTIONS = 10000;
std::atomic<int> active_connections{0};

// On new connection
if (active_connections >= MAX_CONNECTIONS) {
    const char *msg = "HTTP/1.1 503 Service Unavailable\r\n\r\nServer overloaded\n";
    send(client_fd, msg, strlen(msg), 0);
    close(client_fd);
    return;
}

active_connections++;
```

📌 7.2 Request Queue with Timeout

```
struct QueuedRequest {
    int fd;
    std::chrono::steady_clock::time_point queued_at;
};

std::queue<QueuedRequest> request_queue;
const auto MAX_QUEUE_TIME = std::chrono::seconds(5);

// Enqueue request
void enqueue_request(int fd) {
    if (request_queue.size() >= MAX_QUEUE_SIZE) {
        send_error(fd, "Queue full, try again later");
        close(fd);
        return;
    }

    request_queue.push({fd, std::chrono::steady_clock::now()});
}

// Process queue
void process_queue() {
    while (!request_queue.empty()) {
        auto req = request_queue.front();
        request_queue.pop();

        auto now = std::chrono::steady_clock::now();

        if (now - req.queued_at > MAX_QUEUE_TIME) {
            send_error(req.fd, "Request timeout");
            close(req.fd);
            continue;
        }

        handle_request(req.fd);
    }
}
```

📌 7.3 Adaptive Timeouts

```
class AdaptiveTimeout {
private:
    std::chrono::milliseconds base_timeout{1000};
    std::chrono::milliseconds current_timeout{1000};

    double success_rate = 1.0;

public:
    void record_success() {
        success_rate = 0.9 * success_rate + 0.1 * 1.0;

        // Increase timeout if success rate is high
        if (success_rate > 0.95) {
            current_timeout = std::max(base_timeout,
                                       current_timeout - std::chrono::milliseconds(100));
        }
    }

    void record_timeout() {
        success_rate = 0.9 * success_rate + 0.1 * 0.0;

        // Decrease timeout if success rate is low
        if (success_rate < 0.5) {
            current_timeout = std::min(std::chrono::seconds(10),
                                       current_timeout + std::chrono::milliseconds(500));
        }
    }

    std::chrono::milliseconds get_timeout() const {
        return current_timeout;
    }
};
```

📌 7.4 Load Shedding (Drop Requests)

```
double get_cpu_usage();  // Returns 0.0-1.0

void handle_connection(int fd) {
    double cpu = get_cpu_usage();

    if (cpu > 0.90) {
        // Drop 50% of requests
        if (rand() % 2 == 0) {
            const char *msg = "503 Service Unavailable\r\nRetry-After: 60\r\n\r\n";
            send(fd, msg, strlen(msg), 0);
            close(fd);
            return;
        }
    } else if (cpu > 0.80) {
        // Drop 25% of requests
        if (rand() % 4 == 0) {
            const char *msg = "503 Service Unavailable\r\n\r\n";
            send(fd, msg, strlen(msg), 0);
            close(fd);
            return;
        }
    }

    // Process normally
    process_request(fd);
}
```

📌 8. Production-Grade Error Handling

Common Network Errors and How to Handle Them:

Advanced Topics - Zero-Copy, Load Balancing, and Production Patterns

Theory

📌 8.1 EINTR (Interrupted System Call)

Cause: Signal received during syscall (e.g., SIGCHLD).

```
// ❌ WRONG: Treats EINTR as error
int n = recv(fd, buf, sizeof(buf), 0);
if (n < 0) {
    close(fd);  // ❌ Connection dropped on EINTR!
    return;
}

// ✅ CORRECT: Retry on EINTR
int recv_all(int fd, char *buf, size_t len) {
    while (true) {
        int n = recv(fd, buf, len, 0);

        if (n >= 0) return n;

        if (errno == EINTR) {
            continue;  // ✅ Retry
        }

        return -1;  // Real error
    }
}
```

📌 8.2 EPIPE (Broken Pipe)

Cause: Tried to write to closed connection.

```
int n = send(fd, data, len, 0);

if (n < 0 && errno == EPIPE) {
    std::cerr << "Client closed connection\n";
    // Don't treat as error, just close
    close(fd);
    return;
}
```

Important: Use `MSG_NOSIGNAL` to prevent SIGPIPE signal:

```
send(fd, data, len, MSG_NOSIGNAL);  // ✅ No SIGPIPE
```

📌 8.3 ECONNRESET (Connection Reset by Peer)

Cause: Peer sent RST packet (abrupt close).

```
int n = recv(fd, buf, sizeof(buf), 0);

if (n < 0 && errno == ECONNRESET) {
    std::cerr << "Connection reset by peer\n";
    close(fd);
    return;
}
```

📌 8.4 ETIMEDOUT (Operation Timed Out)

Cause: No response within timeout period.

```
// Set socket timeout
struct timeval timeout;
timeout.tv_sec = 30;
timeout.tv_usec = 0;
setsockopt(fd, SOL_SOCKET, SO_RCVTIMEO, &timeout, sizeof(timeout));

int n = recv(fd, buf, sizeof(buf), 0);

if (n < 0 && errno == ETIMEDOUT) {
    std::cerr << "Receive timeout\n";
    // Retry or close
}
```

📌 8.5 EMFILE / ENFILE (Too Many Open Files)

Cause: Process or system FD limit reached.

```
int client_fd = accept(listen_fd, nullptr, nullptr);

if (client_fd < 0) {
    if (errno == EMFILE || errno == ENFILE) {
        std::cerr << "FD limit reached, rejecting connection\n";

        // Temporary strategy: close some idle connections
        close_idle_connections();

        // Or increase limits
        // ulimit -n 100000
    }
}
```

📌 9. Performance Monitoring and Metrics

Essential Metrics for Network Servers:

Advanced Topics - Zero-Copy, Load Balancing, and Production Patterns

Theory

📌 9.1 Request Latency (Percentiles)

```
class LatencyTracker {
private:
    std::vector<double> latencies;
    std::mutex mutex;

public:
    void record(double latency_ms) {
        std::lock_guard<std::mutex> lock(mutex);
        latencies.push_back(latency_ms);

        // Keep only last 10,000 measurements
        if (latencies.size() > 10000) {
            latencies.erase(latencies.begin());
        }
    }

    void print_stats() {
        std::lock_guard<std::mutex> lock(mutex);

        if (latencies.empty()) return;

        std::sort(latencies.begin(), latencies.end());

        size_t p50 = latencies.size() * 50 / 100;
        size_t p95 = latencies.size() * 95 / 100;
        size_t p99 = latencies.size() * 99 / 100;

        std::cout << "Latency:\n"
                  << "  p50: " << latencies[p50] << " ms\n"
                  << "  p95: " << latencies[p95] << " ms\n"
                  << "  p99: " << latencies[p99] << " ms\n";
    }
};
```

📌 9.2 Throughput (Requests/Second)

```
class ThroughputTracker {
private:
    std::atomic<size_t> request_count{0};
    std::chrono::steady_clock::time_point last_print;

public:
    ThroughputTracker() : last_print(std::chrono::steady_clock::now()) {}

    void record_request() {
        request_count++;
    }

    void maybe_print() {
        auto now = std::chrono::steady_clock::now();
        auto elapsed = std::chrono::duration_cast<std::chrono::seconds>(now - last_print);

        if (elapsed.count() >= 10) {
            double rps = request_count / elapsed.count();
            std::cout << "Throughput: " << rps << " req/s\n";

            request_count = 0;
            last_print = now;
        }
    }
};
```

Advanced Topics - Zero-Copy, Load Balancing, and Production Patterns

Theory

📌 9.3 Connection Metrics

```
struct ConnectionMetrics {
    std::atomic<size_t> total_connections{0};
    std::atomic<size_t> active_connections{0};
    std::atomic<size_t> rejected_connections{0};
    std::atomic<size_t> total_bytes_received{0};
    std::atomic<size_t> total_bytes_sent{0};

    void print() {
        std::cout << "=== Connection Metrics ===\n"
                  << "Total: " << total_connections << "\n"
                  << "Active: " << active_connections << "\n"
                  << "Rejected: " << rejected_connections << "\n"
                  << "RX: " << (total_bytes_received / 1024 / 1024) << " MB\n"
                  << "TX: " << (total_bytes_sent / 1024 / 1024) << " MB\n";
    }
};
```

End of THEORY_SECTION

Advanced Topics - Zero-Copy, Load Balancing, and Production Patterns

Edge Cases

📌 1. Zero-Copy with Partial Sends (1/2)

Problem: `sendfile()` can return less than requested bytes. Must handle partial sends.

```
// ❌ WRONG: Assumes sendfile() sends all at once
off_t offset = 0;
sendfile(socket_fd, file_fd, &offset, file_size);  // Might only send part!
```

Why: Socket buffer full, or slow receiver.

Solution: Loop until all bytes sent

```
// ✅ CORRECT: Handle partial sends
off_t offset = 0;
off_t remaining = file_size;

while (remaining > 0) {
    ssize_t sent = sendfile(socket_fd, file_fd, &offset, remaining);

    if (sent < 0) {
        if (errno == EAGAIN || errno == EWOULDBLOCK) {
            // Socket buffer full, wait for EPOLLOUT
            struct epoll_event ev;
            ev.events = EPOLLOUT;
            ev.data.fd = socket_fd;
            epoll_ctl(epfd, EPOLL_CTL_MOD, socket_fd, &ev);

            // Wait for socket to be writable
            struct epoll_event events[1];
            epoll_wait(epfd, events, 1, -1);
            continue;
        }

        // Real error
        perror("sendfile");
        break;
    }

    remaining -= sent;
}

std::cout << "Sent " << (file_size - remaining) << "/" << file_size << " bytes\n";
```

Advanced Topics - Zero-Copy, Load Balancing, and Production Patterns

Edge Cases

📌 1. Zero-Copy with Partial Sends (2/2)

Lesson: Never assume zero-copy functions send all data at once.

Advanced Topics - Zero-Copy, Load Balancing, and Production Patterns

Edge Cases

📌 2. Connection Pool Stale Connections (1/2)

Problem: Connection sits idle in pool, remote server closes it. Next user gets stale connection.

```
// User acquires connection from pool
int fd = pool.acquire();

// Connection was closed by server during idle time
int n = send(fd, "GET / HTTP/1.1\r\n\r\n", 18, 0);
// ❌ EPIPE or ECONNRESET! Connection is dead
```

Solution: Health Check Before Returning

```
class ConnectionPool {
    int acquire() {
        std::lock_guard<std::mutex> lock(pool_mutex);

        while (!available.empty()) {
            int fd = available.front();
            available.pop();

            // ✅ Health check: Try non-blocking send of 0 bytes
            int err = send(fd, nullptr, 0, MSG_DONTWAIT | MSG_NOSIGNAL);

            if (err == 0) {
                // Connection is healthy
                in_use.insert(fd);
                return fd;
            } else if (errno == ENOTCONN || errno == EPIPE) {
                // Connection is dead, close it
                std::cerr << "Stale connection detected, discarding\n";
                close(fd);
                continue;  // Try next connection
            }
        }

        // No healthy connections, create new one
        int fd = create_connection();
        in_use.insert(fd);
        return fd;
    }
};
```

📌 2. Connection Pool Stale Connections (2/2)

Alternative: SO_KEEPALIVE

```
int enable_keepalive(int fd) {
    int opt = 1;
    setsockopt(fd, SOL_SOCKET, SO_KEEPALIVE, &opt, sizeof(opt));

    // Send keepalive probes after 60s idle
    int idle = 60;
    setsockopt(fd, IPPROTO_TCP, TCP_KEEPIDLE, &idle, sizeof(idle));

    // Send probes every 10s
    int interval = 10;
    setsockopt(fd, IPPROTO_TCP, TCP_KEEPINTVL, &interval, sizeof(interval));

    // Close after 3 failed probes
    int count = 3;
    setsockopt(fd, IPPROTO_TCP, TCP_KEEPCNT, &count, sizeof(count));

    return 0;
}
```

Lesson: Always validate pooled connections before use.

Advanced Topics - Zero-Copy, Load Balancing, and Production Patterns

Edge Cases

📌 3. Load Balancer Backend Failure Detection (1/4)

Problem: Backend server crashes, load balancer keeps sending traffic to it.

```
Request 1 → Backend A ✅
Request 2 → Backend B (CRASHED!) ❌
Request 3 → Backend C ✅
Request 4 → Backend B (STILL CRASHED!) ❌
```

Advanced Topics - Zero-Copy, Load Balancing, and Production Patterns

Edge Cases

📌 3. Load Balancer Backend Failure Detection (2/4)

Solution: Circuit Breaker Pattern

```
class CircuitBreaker {
private:
    enum class State { CLOSED, OPEN, HALF_OPEN };

    struct Backend {
        std::string host;
        State state = State::CLOSED;
        int failure_count = 0;
        std::chrono::steady_clock::time_point opened_at;
    };

    std::vector<Backend> backends;

    const int FAILURE_THRESHOLD = 5;
    const auto TIMEOUT = std::chrono::seconds(30);

public:
    std::string select_backend() {
        auto now = std::chrono::steady_clock::now();

        for (auto& backend : backends) {
            // Check if circuit should transition to HALF_OPEN
            if (backend.state == State::OPEN) {
                if (now - backend.opened_at > TIMEOUT) {
                    backend.state = State::HALF_OPEN;
                    std::cout << "Circuit HALF_OPEN for " << backend.host << "\n";
                }
            }

            // Select healthy or half-open backend
            if (backend.state == State::CLOSED || backend.state == State::HALF_OPEN) {
                return backend.host;
            }
        }

        // All backends down!
        throw std::runtime_error("No healthy backends");
    }

// ...continues on next slide
```

📌 3. Load Balancer Backend Failure Detection (3/4)

```
// ...continued from previous slide
    void record_success(const std::string& host) {
        for (auto& backend : backends) {
            if (backend.host == host) {
                if (backend.state == State::HALF_OPEN) {
                    // Test request succeeded, close circuit
                    backend.state = State::CLOSED;
                    backend.failure_count = 0;
                    std::cout << "Circuit CLOSED for " << host << "\n";
                }
                break;
            }
        }
    }

    void record_failure(const std::string& host) {
        for (auto& backend : backends) {
            if (backend.host == host) {
                backend.failure_count++;

                if (backend.failure_count >= FAILURE_THRESHOLD) {
                    backend.state = State::OPEN;
                    backend.opened_at = std::chrono::steady_clock::now();
                    std::cout << "Circuit OPEN for " << host
                              << " (failures: " << backend.failure_count << ")\n";
                }
                break;
            }
        }
    }
};

// Usage
CircuitBreaker cb;

try {
    std::string backend = cb.select_backend();

    if (send_request_to(backend)) {
        cb.record_success(backend);
    } else {
// ...continues on next slide
```

📌 3. Load Balancer Backend Failure Detection (4/4)

[Code continues across slides]

```
// ...continued from previous slide
        cb.record_failure(backend);
    }
} catch (const std::exception& e) {
    std::cerr << "All backends unavailable: " << e.what() << "\n";
    send_503_error();
}
```

Circuit States:

- **CLOSED:** Normal operation, backend is healthy
- **OPEN:** Backend is failing, don't send traffic
- **HALF_OPEN:** Testing if backend recovered (send 1 request)

Lesson: Don't blindly route to failed backends.

📌 4. Protocol Version Mismatch Recovery (1/3)

Problem: Client sends v2 protocol, server only supports v1. How to fail gracefully?

Bad Approach:

```
// ❌ WRONG: Just close connection silently
if (client_version > SERVER_VERSION) {
    close(fd);  // Client has no idea why connection was closed
}
```

📌 4. Protocol Version Mismatch Recovery (2/3)

Good Approach: Send Error Response in Compatible Format

```
struct HandshakeResponse {
    uint8_t magic[4] = {'M', 'Y', 'P', 'X'};
    uint16_t server_version_major;
    uint16_t server_version_minor;
    uint16_t status;  // 0 = OK, 1 = VERSION_TOO_NEW, 2 = VERSION_TOO_OLD
    char error_message[64];
};

void handle_handshake(int fd) {
    Handshake hs;
    recv(fd, &hs, sizeof(hs), MSG_WAITALL);

    HandshakeResponse resp;
    resp.server_version_major = htons(1);
    resp.server_version_minor = htons(0);

    uint16_t client_major = ntohs(hs.version_major);

    if (client_major > 1) {
        // Client too new
        resp.status = htons(1);
        snprintf(resp.error_message, sizeof(resp.error_message),
                "Server supports v1.x, client is v%d.x", client_major);

        send(fd, &resp, sizeof(resp), 0);
        close(fd);
        return;
    }

    if (client_major < 1) {
        // Client too old
        resp.status = htons(2);
        snprintf(resp.error_message, sizeof(resp.error_message),
                "Server requires v1.x, client is v%d.x", client_major);

        send(fd, &resp, sizeof(resp), 0);
        close(fd);
        return;
    }
// ...continues on next slide
```

📌 4. Protocol Version Mismatch Recovery (3/3)

[Code continues across slides]

```
// ...continued from previous slide

    // Version OK
    resp.status = 0;
    strcpy(resp.error_message, "OK");
    send(fd, &resp, sizeof(resp), 0);
}
```

Lesson: Always send error responses in a format the client can understand.

Advanced Topics - Zero-Copy, Load Balancing, and Production Patterns

Edge Cases

📌 5. Backpressure Deadlock (1/3)

Problem: Two servers sending data to each other, both hit backpressure, neither can drain the other's buffer.

```
Server A write buffer full → stops reading from Server B
Server B write buffer full → stops reading from Server A
   ↓
DEADLOCK! Neither can make progress.
```

Visual:

```
Time   | Server A                          | Server B
------|-----------------------------------|-----------------------------------
T1     | Sends 10 MB → B                   | Sends 10 MB → A
T2     | Write buffer full (stop reading)| Write buffer full (stop reading)
T3     | Waiting for A to drain...         | Waiting for B to drain...
T4     | ❌ DEADLOCK                       | ❌ DEADLOCK
```

📌 5. Backpressure Deadlock (2/3)

Solution: Always Prioritize Reading Over Writing

```
// ❌ WRONG: Only monitor EPOLLOUT when backpressured
if (write_queue[peer_fd].size() > MAX_QUEUE_SIZE) {
    ev.events = EPOLLOUT;  // ❌ Stopped reading!
    epoll_ctl(epfd, EPOLL_CTL_MOD, peer_fd, &ev);
}

// ✅ CORRECT: Always monitor EPOLLIN, even during backpressure
if (write_queue[peer_fd].size() > MAX_QUEUE_SIZE) {
    ev.events = EPOLLIN | EPOLLOUT;  // ✅ Still reading
    epoll_ctl(epfd, EPOLL_CTL_MOD, peer_fd, &ev);

    // Apply backpressure upstream (stop accepting new connections)
    epoll_ctl(epfd, EPOLL_CTL_DEL, listen_fd, nullptr);
}

// When write queue drains
if (write_queue[peer_fd].empty()) {
    ev.events = EPOLLIN;
    epoll_ctl(epfd, EPOLL_CTL_MOD, peer_fd, &ev);

    // Resume accepting connections
    ev.events = EPOLLIN;
    ev.data.fd = listen_fd;
    epoll_ctl(epfd, EPOLL_CTL_ADD, listen_fd, &ev);
}
```

📌 5. Backpressure Deadlock (3/3)

Alternative: Use Larger Buffers or Drop Data

```
if (write_queue[fd].size() > MAX_QUEUE_SIZE) {
    // Option 1: Increase buffer (temporary relief)
    if (write_queue[fd].size() < ABSOLUTE_MAX_QUEUE_SIZE) {
        write_queue[fd].push(data);
        std::cerr << "Warning: Large write queue for FD " << fd << "\n";
    } else {
        // Option 2: Drop oldest data (lossy but prevents deadlock)
        std::cerr << "Dropping data due to backpressure on FD " << fd << "\n";
        write_queue[fd].pop_front();
        write_queue[fd].push(data);
    }
}
```

Lesson: Never completely stop reading during backpressure. Always drain incoming data to prevent deadlocks.

Advanced Topics - Zero-Copy, Load Balancing, and Production Patterns

Code Examples

📌 1. Production HTTP File Server with sendfile() (1/6)

Why: Demonstrates zero-copy file serving with proper error handling, partial sends, and epoll integration.

Compile & Test:

Expected Output:

```
#include <sys/epoll.h>
#include <sys/sendfile.h>
#include <sys/socket.h>
#include <sys/stat.h>
#include <netinet/in.h>
#include <fcntl.h>
#include <unistd.h>
#include <cstring>
#include <iostream>
#include <string>
#include <sstream>

struct FileTransfer {
    int file_fd;
    off_t offset;
    off_t remaining;
    std::string filepath;
};

std::unordered_map<int, FileTransfer> active_transfers;

void serve_file(int client_fd, const std::string& filepath) {
    int file_fd = open(filepath.c_str(), O_RDONLY);

    if (file_fd < 0) {
        const char *resp = "HTTP/1.1 404 Not Found\r\nContent-Length: 9\r\n\r\nNot Found";
        send(client_fd, resp, strlen(resp), 0);
        return;
    }

    // Get file size
    struct stat st;
    fstat(file_fd, &st);

// ...continues on next slide
```

Advanced Topics - Zero-Copy, Load Balancing, and Production Patterns

Code Examples

📌 1. Production HTTP File Server with sendfile() (2/6)

```
// ...continued from previous slide
    // Send HTTP headers
    std::ostringstream headers;
    headers << "HTTP/1.1 200 OK\r\n"
            << "Content-Length: " << st.st_size << "\r\n"
            << "Connection: close\r\n"
            << "\r\n";
    std::string header_str = headers.str();
    send(client_fd, header_str.data(), header_str.size(), 0);

    // Start transfer
    FileTransfer transfer;
    transfer.file_fd = file_fd;
    transfer.offset = 0;
    transfer.remaining = st.st_size;
    transfer.filepath = filepath;

    active_transfers[client_fd] = transfer;

    std::cout << "[FD " << client_fd << "] Starting transfer: "
              << filepath << " (" << st.st_size << " bytes)\n";
}

bool continue_transfer(int epfd, int client_fd) {
    auto& transfer = active_transfers[client_fd];

    // Try to send up to 64 KB at a time
    const size_t CHUNK_SIZE = 65536;
    size_t to_send = std::min((size_t)transfer.remaining, CHUNK_SIZE);

    ssize_t sent = sendfile(client_fd, transfer.file_fd, &transfer.offset, to_send);

    if (sent < 0) {
        if (errno == EAGAIN || errno == EWOULDBLOCK) {
            // Socket buffer full, wait for EPOLLOUT
            struct epoll_event ev;
            ev.events = EPOLLOUT;
            ev.data.fd = client_fd;
            epoll_ctl(epfd, EPOLL_CTL_MOD, client_fd, &ev);
            return false;  // Not complete yet
        }
// ...continues on next slide
```

Advanced Topics - Zero-Copy, Load Balancing, and Production Patterns

Code Examples

📌 1. Production HTTP File Server with sendfile() (3/6)

```
// ...continued from previous slide

        // Error
        perror("sendfile");
        close(transfer.file_fd);
        active_transfers.erase(client_fd);
        return true;  // Transfer failed, close connection
    }

    transfer.remaining -= sent;

    if (transfer.remaining == 0) {
        // Transfer complete!
        std::cout << "[FD " << client_fd << "] Transfer complete: "
                  << transfer.filepath << "\n";

        close(transfer.file_fd);
        active_transfers.erase(client_fd);
        return true;  // Done
    }

    // More data to send
    return false;
}

int main() {
    // Create server socket
    int server_fd = socket(AF_INET, SOCK_STREAM, 0);
    int opt = 1;
    setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));

    sockaddr_in addr{};
    addr.sin_family = AF_INET;
    addr.sin_addr.s_addr = INADDR_ANY;
    addr.sin_port = htons(8080);

    bind(server_fd, (sockaddr*)&addr, sizeof(addr));
    listen(server_fd, 128);

    // Create epoll
    int epfd = epoll_create1(EPOLL_CLOEXEC);
// ...continues on next slide
```

Advanced Topics - Zero-Copy, Load Balancing, and Production Patterns

Code Examples

📌 1. Production HTTP File Server with sendfile() (4/6)

```
// ...continued from previous slide

    // Add server socket
    struct epoll_event ev;
    ev.events = EPOLLIN;
    ev.data.fd = server_fd;
    epoll_ctl(epfd, EPOLL_CTL_ADD, server_fd, &ev);

    std::cout << "Zero-copy HTTP file server on :8080\n";
    std::cout << "Try: curl http://localhost:8080/test.txt\n\n";

    struct epoll_event events[128];

    while (true) {
        int nready = epoll_wait(epfd, events, 128, -1);

        for (int i = 0; i < nready; i++) {
            int fd = events[i].data.fd;

            if (fd == server_fd) {
                // Accept new connection
                int client_fd = accept(server_fd, nullptr, nullptr);

                // Set non-blocking
                int flags = fcntl(client_fd, F_GETFL, 0);
                fcntl(client_fd, F_SETFL, flags | O_NONBLOCK);

                // Add to epoll
                struct epoll_event cev;
                cev.events = EPOLLIN;
                cev.data.fd = client_fd;
                epoll_ctl(epfd, EPOLL_CTL_ADD, client_fd, &cev);

                std::cout << "[FD " << client_fd << "] New connection\n";

            } else if (events[i].events & EPOLLIN) {
                // Read HTTP request
                char buf[4096];
                int n = recv(fd, buf, sizeof(buf), 0);

                if (n <= 0) {
// ...continues on next slide
```

Advanced Topics - Zero-Copy, Load Balancing, and Production Patterns

Code Examples

📌 1. Production HTTP File Server with sendfile() (5/6)

```
// ...continued from previous slide
                    epoll_ctl(epfd, EPOLL_CTL_DEL, fd, nullptr);
                    close(fd);
                    std::cout << "[FD " << fd << "] Connection closed\n";
                    continue;
                }

                // Parse request (simple: just extract path)
                std::string request(buf, n);
                size_t path_start = request.find(' ') + 1;
                size_t path_end = request.find(' ', path_start);
                std::string path = request.substr(path_start, path_end - path_start);

                if (path == "/") path = "/index.html";

                std::string filepath = "./www" + path;

                std::cout << "[FD " << fd << "] GET " << path << "\n";

                // Start file transfer
                serve_file(fd, filepath);

                // Immediately try to send some data
                if (!continue_transfer(epfd, fd)) {
                    // Transfer not complete, wait for EPOLLOUT
                    std::cout << "[FD " << fd << "] Waiting for EPOLLOUT\n";
                }

            } else if (events[i].events & EPOLLOUT) {
                // Continue file transfer
                if (continue_transfer(epfd, fd)) {
                    // Transfer complete, close connection
                    epoll_ctl(epfd, EPOLL_CTL_DEL, fd, nullptr);
                    close(fd);
                    std::cout << "[FD " << fd << "] Connection closed\n";
                }
            }
        }
    }

    close(epfd);
// ...continues on next slide
```

Advanced Topics - Zero-Copy, Load Balancing, and Production Patterns

Code Examples

📌 1. Production HTTP File Server with sendfile() (6/6)

[Code continues across slides]

```
// ...continued from previous slide
    close(server_fd);

    return 0;
}
```

```
['**Expected Output:**']
```

Advanced Topics - Zero-Copy, Load Balancing, and Production Patterns

Code Examples

📌 2. Connection Pool with Health Checks (1/7)

Why: Demonstrates production-grade connection pooling with stale connection detection and automatic recovery.

Expected Output:

```
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <netinet/tcp.h>
#include <unistd.h>
#include <fcntl.h>
#include <errno.h>
#include <cstring>
#include <iostream>
#include <string>
#include <queue>
#include <set>
#include <mutex>
#include <chrono>
#include <unordered_map>

class AdvancedConnectionPool {
private:
    std::string host;
    int port;

    std::queue<int> available;
    std::set<int> in_use;
    std::unordered_map<int, std::chrono::steady_clock::time_point> last_used;

    std::mutex pool_mutex;

    int max_connections = 10;
    int idle_timeout_sec = 60;

    std::atomic<size_t> total_created{0};
    std::atomic<size_t> total_reused{0};
    std::atomic<size_t> total_stale_detected{0};

public:
    AdvancedConnectionPool(const std::string& h, int p, int max_conns = 10)
// ...continues on next slide
```

Advanced Topics - Zero-Copy, Load Balancing, and Production Patterns

Code Examples

📌 2. Connection Pool with Health Checks (2/7)

```
// ...continued from previous slide
        : host(h), port(p), max_connections(max_conns) {}

    ~AdvancedConnectionPool() {
        std::lock_guard<std::mutex> lock(pool_mutex);

        // Close all connections
        while (!available.empty()) {
            close(available.front());
            available.pop();
        }

        for (int fd : in_use) {
            close(fd);
        }
    }

    int acquire() {
        std::lock_guard<std::mutex> lock(pool_mutex);

        // Try to reuse existing connection
        while (!available.empty()) {
            int fd = available.front();
            available.pop();

            // Health check: Send 0 bytes to check if connection is alive
            int err = send(fd, nullptr, 0, MSG_DONTWAIT | MSG_NOSIGNAL);

            if (err == 0) {
                // Connection is healthy
                in_use.insert(fd);
                total_reused++;

                std::cout << "[Pool] Reused connection FD " << fd
                          << " (reuse rate: " << (total_reused * 100 / total_created) << "%)\n";

                return fd;
            } else {
                // Connection is stale
                std::cerr << "[Pool] Stale connection detected: FD " << fd
                          << " (errno: " << strerror(errno) << ")\n";
// ...continues on next slide
```

Advanced Topics - Zero-Copy, Load Balancing, and Production Patterns

Code Examples

📌 2. Connection Pool with Health Checks (3/7)

```
// ...continued from previous slide

                close(fd);
                last_used.erase(fd);
                total_stale_detected++;
                continue;
            }
        }

        // No available connections, create new one
        if (in_use.size() < max_connections) {
            int fd = create_connection();

            if (fd >= 0) {
                in_use.insert(fd);
                total_created++;

                std::cout << "[Pool] Created new connection FD " << fd
                          << " (total: " << total_created << ")\n";

                return fd;
            }
        }

        // Pool exhausted
        std::cerr << "[Pool] Pool exhausted (in_use: " << in_use.size()
                  << ", max: " << max_connections << ")\n";
        return -1;
    }

    void release(int fd) {
        std::lock_guard<std::mutex> lock(pool_mutex);

        in_use.erase(fd);
        available.push(fd);
        last_used[fd] = std::chrono::steady_clock::now();

        std::cout << "[Pool] Released FD " << fd
                  << " (available: " << available.size() << ")\n";
    }

// ...continues on next slide
```

Advanced Topics - Zero-Copy, Load Balancing, and Production Patterns

Code Examples

📌 2. Connection Pool with Health Checks (4/7)

```
// ...continued from previous slide
    void cleanup_idle() {
        std::lock_guard<std::mutex> lock(pool_mutex);

        auto now = std::chrono::steady_clock::now();
        size_t cleaned = 0;

        while (!available.empty()) {
            int fd = available.front();

            auto it = last_used.find(fd);
            if (it == last_used.end()) break;

            auto age = std::chrono::duration_cast<std::chrono::seconds>(now - it->second);

            if (age.count() > idle_timeout_sec) {
                available.pop();
                close(fd);
                last_used.erase(fd);
                cleaned++;
            } else {
                break;  // Queue is time-ordered
            }
        }

        if (cleaned > 0) {
            std::cout << "[Pool] Cleaned " << cleaned << " idle connections\n";
        }
    }

    void print_stats() {
        std::lock_guard<std::mutex> lock(pool_mutex);

        std::cout << "\n=== Connection Pool Stats ===\n"
                  << "Total created: " << total_created << "\n"
                  << "Total reused: " << total_reused << "\n"
                  << "Stale detected: " << total_stale_detected << "\n"
                  << "Currently available: " << available.size() << "\n"
                  << "Currently in use: " << in_use.size() << "\n"
                  << "Reuse rate: " << (total_reused * 100 / std::max(total_created, 1UL)) << "%\n";
    }
// ...continues on next slide
```

Advanced Topics - Zero-Copy, Load Balancing, and Production Patterns

Code Examples

📌 2. Connection Pool with Health Checks (5/7)

```
// ...continued from previous slide

private:
    int create_connection() {
        int fd = socket(AF_INET, SOCK_STREAM, 0);
        if (fd < 0) {
            perror("socket");
            return -1;
        }

        // Enable TCP keepalive
        int opt = 1;
        setsockopt(fd, SOL_SOCKET, SO_KEEPALIVE, &opt, sizeof(opt));

        int idle = 30;  // Start keepalive after 30s idle
        setsockopt(fd, IPPROTO_TCP, TCP_KEEPIDLE, &idle, sizeof(idle));

        int interval = 10;  // Send probes every 10s
        setsockopt(fd, IPPROTO_TCP, TCP_KEEPINTVL, &interval, sizeof(interval));

        int count = 3;  // Close after 3 failed probes
        setsockopt(fd, IPPROTO_TCP, TCP_KEEPCNT, &count, sizeof(count));

        // Connect
        sockaddr_in addr{};
        addr.sin_family = AF_INET;
        addr.sin_port = htons(port);
        inet_pton(AF_INET, host.c_str(), &addr.sin_addr);

        if (connect(fd, (sockaddr*)&addr, sizeof(addr)) < 0) {
            perror("connect");
            close(fd);
            return -1;
        }

        // Set non-blocking
        int flags = fcntl(fd, F_GETFL, 0);
        fcntl(fd, F_SETFL, flags | O_NONBLOCK);

        return fd;
    }
// ...continues on next slide
```

Advanced Topics - Zero-Copy, Load Balancing, and Production Patterns

Code Examples

📌 2. Connection Pool with Health Checks (6/7)

[Code continues across slides]

```
// ...continued from previous slide
};

// Usage Example
int main() {
    AdvancedConnectionPool pool("httpbin.org", 80, 5);

    // Simulate 100 requests
    for (int i = 0; i < 100; i++) {
        int fd = pool.acquire();

        if (fd < 0) {
            std::cerr << "Failed to acquire connection\n";
            continue;
        }

        // Send HTTP request
        const char *request = "GET /get HTTP/1.1\r\nHost: httpbin.org\r\nConnection: keep-alive\r\n\r\n";
        send(fd, request, strlen(request), 0);

        // Read response (simplified)
        char buf[4096];
        recv(fd, buf, sizeof(buf), 0);

        // Return to pool
        pool.release(fd);

        // Periodic cleanup
        if (i % 10 == 0) {
            pool.cleanup_idle();
        }

        // Small delay
        usleep(100000);  // 100ms
    }

    pool.print_stats();

    return 0;
}
```

Advanced Topics - Zero-Copy, Load Balancing, and Production Patterns

Code Examples

📌 2. Connection Pool with Health Checks (7/7)

```
['[Pool] Created new connection FD 3 (total: 1)\n[Pool] Released FD 3 (available: 1)\n[Pool] Reused connection FD 3 (re
```

Advanced Topics - Zero-Copy, Load Balancing, and Production Patterns

Code Examples

📌 3. Reactor Pattern Implementation (1/3)

Why: Clean separation of I/O handling and business logic using event handlers.

```
#include <sys/epoll.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <fcntl.h>
#include <unistd.h>
#include <functional>
#include <unordered_map>
#include <iostream>

class Reactor {
private:
    int epfd;
    std::unordered_map<int, std::function<void(uint32_t)>> handlers;
    bool running = true;

public:
    Reactor() {
        epfd = epoll_create1(EPOLL_CLOEXEC);
    }

    ~Reactor() {
        close(epfd);
    }

    void register_handler(int fd, uint32_t events, std::function<void(uint32_t)> handler) {
        handlers[fd] = handler;

        struct epoll_event ev;
        ev.events = events;
        ev.data.fd = fd;
        epoll_ctl(epfd, EPOLL_CTL_ADD, fd, &ev);

        std::cout << "[Reactor] Registered FD " << fd << "\n";
    }

    void unregister_handler(int fd) {
        epoll_ctl(epfd, EPOLL_CTL_DEL, fd, nullptr);
// ...continues on next slide
```

Advanced Topics - Zero-Copy, Load Balancing, and Production Patterns

Code Examples

📌 3. Reactor Pattern Implementation (2/3)

```
// ...continued from previous slide
        handlers.erase(fd);

        std::cout << "[Reactor] Unregistered FD " << fd << "\n";
    }

    void run() {
        struct epoll_event events[128];

        while (running) {
            int n = epoll_wait(epfd, events, 128, 1000);

            for (int i = 0; i < n; i++) {
                int fd = events[i].data.fd;
                uint32_t ev = events[i].events;

                if (handlers.count(fd)) {
                    handlers[fd](ev);
                }
            }
        }
    }

    void stop() {
        running = false;
    }
};

int main() {
    Reactor reactor;

    int server_fd = socket(AF_INET, SOCK_STREAM, 0);
    int opt = 1;
    setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));

    sockaddr_in addr{};
    addr.sin_family = AF_INET;
    addr.sin_addr.s_addr = INADDR_ANY;
    addr.sin_port = htons(8080);

    bind(server_fd, (sockaddr*)&addr, sizeof(addr));
// ...continues on next slide
```

Advanced Topics - Zero-Copy, Load Balancing, and Production Patterns

Code Examples

📌 3. Reactor Pattern Implementation (3/3)

[Code continues across slides]

```
// ...continued from previous slide
    listen(server_fd, 128);

    reactor.register_handler(server_fd, EPOLLIN, [&](uint32_t events) {
        int client_fd = accept(server_fd, nullptr, nullptr);

        int flags = fcntl(client_fd, F_GETFL, 0);
        fcntl(client_fd, F_SETFL, flags | O_NONBLOCK);

        std::cout << "[Server] Accepted FD " << client_fd << "\n";

        reactor.register_handler(client_fd, EPOLLIN, [&reactor, client_fd](uint32_t events) {
            char buf[1024];
            int n = recv(client_fd, buf, sizeof(buf), 0);

            if (n <= 0) {
                reactor.unregister_handler(client_fd);
                close(client_fd);
                std::cout << "[Client] FD " << client_fd << " disconnected\n";
            } else {
                send(client_fd, buf, n, 0);
            }
        });
    });

    std::cout << "Reactor echo server on :8080\n";
    reactor.run();

    close(server_fd);
    return 0;
}
```

Advanced Topics - Zero-Copy, Load Balancing, and Production Patterns

Code Examples

📌 4. Load Balancer with Multiple Strategies (1/6)

Why: Compare round-robin, least-connections, and circuit-breaker patterns.

Break 1 Complete! Topic 6 Break 1 now contains:
- ✅ THEORY_SECTION (9 sections, 1,555 lines)
- ✅ 5 EDGE_CASES
- ✅ 4 CODE_EXAMPLES (sendfile, connection pool, reactor, load balancer)

Next: Break 2 (INTERVIEW_QA) and Break 3 (PRACTICE_TASKS + QUICK_REFERENCE)

```
#include <string>
#include <vector>
#include <atomic>
#include <chrono>
#include <iostream>

class MultiStrategyLoadBalancer {
private:
    struct Backend {
        std::string host;
        std::atomic<int> active_conns{0};
        int weight = 1;

        enum class State { HEALTHY, DEGRADED, DOWN };
        State state = State::HEALTHY;
        int failure_count = 0;
        std::chrono::steady_clock::time_point last_failure;
    };

    std::vector<Backend> backends;
    std::atomic<size_t> round_robin_index{0};

public:
    enum class Strategy { ROUND_ROBIN, LEAST_CONNECTIONS, WEIGHTED, CIRCUIT_BREAKER };

    void add_backend(const std::string& host, int weight = 1) {
        Backend b;
        b.host = host;
        b.weight = weight;
// ...continues on next slide
```

Advanced Topics - Zero-Copy, Load Balancing, and Production Patterns

Code Examples

📌 4. Load Balancer with Multiple Strategies (2/6)

```
// ...continued from previous slide
        backends.push_back(b);
    }

    std::string select(Strategy strategy) {
        switch (strategy) {
            case Strategy::ROUND_ROBIN:
                return select_round_robin();
            case Strategy::LEAST_CONNECTIONS:
                return select_least_connections();
            case Strategy::WEIGHTED:
                return select_weighted();
            case Strategy::CIRCUIT_BREAKER:
                return select_circuit_breaker();
        }
        return backends[0].host;
    }

    void record_success(const std::string& host) {
        for (auto& b : backends) {
            if (b.host == host) {
                if (b.state == Backend::State::DEGRADED) {
                    b.state = Backend::State::HEALTHY;
                    b.failure_count = 0;
                    std::cout << "[LB] Backend " << host << " recovered\n";
                }
                break;
            }
        }
    }

    void record_failure(const std::string& host) {
        for (auto& b : backends) {
            if (b.host == host) {
                b.failure_count++;
                b.last_failure = std::chrono::steady_clock::now();

                if (b.failure_count >= 5) {
                    b.state = Backend::State::DOWN;
                    std::cout << "[LB] Backend " << host << " marked DOWN\n";
                } else if (b.failure_count >= 3) {
// ...continues on next slide
```

Advanced Topics - Zero-Copy, Load Balancing, and Production Patterns

Code Examples

📌 4. Load Balancer with Multiple Strategies (3/6)

```
// ...continued from previous slide
                    b.state = Backend::State::DEGRADED;
                }
                break;
            }
        }
    }

    void acquire_connection(const std::string& host) {
        for (auto& b : backends) {
            if (b.host == host) {
                b.active_conns++;
                break;
            }
        }
    }

    void release_connection(const std::string& host) {
        for (auto& b : backends) {
            if (b.host == host) {
                b.active_conns--;
                break;
            }
        }
    }

private:
    std::string select_round_robin() {
        size_t idx = round_robin_index.fetch_add(1) % backends.size();
        return backends[idx].host;
    }

    std::string select_least_connections() {
        Backend *selected = &backends[0];
        int min_conns = selected->active_conns;

        for (auto& b : backends) {
            if (b.active_conns < min_conns) {
                selected = &b;
                min_conns = b.active_conns;
            }
// ...continues on next slide
```

Advanced Topics - Zero-Copy, Load Balancing, and Production Patterns

Code Examples

📌 4. Load Balancer with Multiple Strategies (4/6)

```
// ...continued from previous slide
        }

        return selected->host;
    }

    std::string select_weighted() {
        int total_weight = 0;
        for (auto& b : backends) {
            total_weight += b.weight;
        }

        int random = rand() % total_weight;
        int cumulative = 0;

        for (auto& b : backends) {
            cumulative += b.weight;
            if (random < cumulative) {
                return b.host;
            }
        }

        return backends[0].host;
    }

    std::string select_circuit_breaker() {
        auto now = std::chrono::steady_clock::now();

        for (auto& b : backends) {
            if (b.state == Backend::State::HEALTHY) {
                return b.host;
            }

            if (b.state == Backend::State::DOWN) {
                auto elapsed = std::chrono::duration_cast<std::chrono::seconds>(
                    now - b.last_failure);

                if (elapsed.count() > 30) {
                    b.state = Backend::State::DEGRADED;
                    std::cout << "[LB] Trying to recover " << b.host << "\n";
                    return b.host;
// ...continues on next slide
```

Advanced Topics - Zero-Copy, Load Balancing, and Production Patterns

Code Examples

📌 4. Load Balancer with Multiple Strategies (5/6)

```
// ...continued from previous slide
                }
            }
        }

        throw std::runtime_error("All backends down");
    }
};

int main() {
    MultiStrategyLoadBalancer lb;
    lb.add_backend("server1.com", 5);
    lb.add_backend("server2.com", 3);
    lb.add_backend("server3.com", 1);

    std::cout << "=== Round Robin ===\n";
    for (int i = 0; i < 9; i++) {
        std::cout << "Request " << i << " → "
                  << lb.select(MultiStrategyLoadBalancer::Strategy::ROUND_ROBIN) << "\n";
    }

    std::cout << "\n=== Least Connections ===\n";
    lb.acquire_connection("server1.com");
    lb.acquire_connection("server1.com");
    lb.acquire_connection("server2.com");

    for (int i = 0; i < 3; i++) {
        std::cout << "Request → "
                  << lb.select(MultiStrategyLoadBalancer::Strategy::LEAST_CONNECTIONS) << "\n";
    }

    std::cout << "\n=== Circuit Breaker ===\n";
    for (int i = 0; i < 6; i++) {
        lb.record_failure("server1.com");
    }

    try {
        std::cout << lb.select(MultiStrategyLoadBalancer::Strategy::CIRCUIT_BREAKER) << "\n";
    } catch (const std::exception& e) {
        std::cerr << "Error: " << e.what() << "\n";
    }
// ...continues on next slide
```

📌 4. Load Balancer with Multiple Strategies (6/6)

[Code continues across slides]

```
// ...continued from previous slide

    return 0;
}
```

What You Learned

Key Takeaways

- ✓ Overview
- ✓ Zero-Copy I/O: Eliminating Memory Copies
- ✓ Kernel-Level File Transfer
- ✓ Zero-Copy Between FDs

→ *Swipe to learn* →